

CSE 565 Performance Testing Project

Chase Coleman

Arizona State University

CSE 565: Software Verification and Validation

Performance Testing Project

October 4, 2026

1 Task 1: Identification of the Performance Testing Framework

Locust 2.46.6 was selected as the performance-testing framework. Locust is a Python framework in which a user class defines the work performed by a simulated user. It records request count, failures, response times, response-time percentiles, and requests per second (RPS). The framework can be run from the command line in headless mode and can create HTML and CSV reports. The Locust training module used for the framework setup is *Your first test*: <https://docs.locust.io/en/stable/quickstart.html>.

The instructor-provided `load_test_tasks.py` file was preserved without modification. The separate `locustfile.py` defines a Locust user workflow with the same six workload categories: data processing, simulated database query, API request, file I/O, computation, and logging. A controlled local profile was used so that the 10,000-user test could be executed without directing high-volume traffic to the public API in the starter script.

2 Task 2: Test Case Design

Each Locust user repeatedly executes the six workload categories in one workflow. Locust records a separate **TASK** result for each category. The controlled API task returns a simulated response; the remaining operations perform local processing, waiting, file I/O, computation, and logging. The three baseline cases use the same standard profile and a 30-second observation window.

Table 1: Baseline Locust test cases.

Test	Load level	Users	Spawn rate	Duration	Profile
Test 1	Baseline load	100	100 users/s	30 s	Standard
Test 2	Medium load	1,000	1,000 users/s	30 s	Standard
Test 3	High load	10,000	10,000 users/s	30 s	Standard

The following command is implemented by `run_locust_scenarios.sh`. It invokes Locust in headless mode and saves an HTML report, statistics CSV, statistics-history CSV, failures CSV, exceptions CSV, and console log for each load level.

```
CSE565_PROFILE=standard CSE565_API_MODE=simulated
CSE565_DURATION_SECONDS=30 .venv/bin/locust -f locustfile.py --headless
--users 100 --spawn-rate 100 --stop-timeout 15
--csv locust_results/users-100/locust
--html locust_results/users-100/locust_report.html
```

Figures 1 and 2 show the implemented scenario script and Locust workflow. The script invokes the same test process at 100, 1,000, and 10,000 immediately spawned users, and the workflow records each of the six workload categories separately.

```
CSE565 > Performance Testing Assignment > $ run_locust_scenarios.sh
1  #!/usr/bin/env bash
2  # Run the required Locust scenarios and preserve raw framework reports.
3  set -euo pipefail
4
5  project_dir="$(cd "$(dirname "$0")" && pwd)"
6  cd "$project_dir"
7
8  timestamp="$(date +%Y%m%d-%H%M%S)"
9  results_root="locust_results/$timestamp"
10 mkdir -p "$results_root"
11
12 run_scenario() {
13     local users="$1"
14     local spawn_rate="$2"
15     local run_seconds="$3"
16     local scenario_dir="$results_root/users-${users}"
17     mkdir -p "$scenario_dir/artifacts"
18
19     echo "Running Locust: users=$users spawn_rate=$spawn_rate duration=${run_seconds}s"
20     CSE565_PROFILE=standard \
21     CSE565_API_MODE=simulated \
22     CSE565_ARTIFACT_DIR="$scenario_dir/artifacts" \
23     CSE565_DURATION_SECONDS="$run_seconds" \
24     .venv/bin/locust -f locustfile.py --headless \
25     --users "$users" --spawn-rate "$spawn_rate" --stop-timeout 15 \
26     --csv "$scenario_dir/locust" --csv-full-history \
27     --html "$scenario_dir/locust_report.html" \
28     --only-summary \
29     2>&1 | tee "$scenario_dir/locust_console.log"
30 }
31
32 # Immediate-spawn Task 3 tests: each requested user count is scheduled at once.
33 run_scenario 100 100 30
34 run_scenario 1000 1000 30
35 run_scenario 10000 10000 30
36
37 echo "Locust reports saved in: $results_root"
38
```

Figure 1: Scenario script used to execute the 100-, 1,000-, and 10,000-user baseline tests.

```

42 PROFILES = {
43     # Used for the required 100, 1,000, and 10,000-user comparison.
44     "standard": WorkloadProfile(500, 0.005, 0.010, 1_024, 500),
45     # Intended for Task 5 after the baseline results have been reviewed.
46     "stress": WorkloadProfile(20_000, 0.050, 0.075, 32_768, 10_000),
47 }
48
49 PROFILE_NAME = os.getenv("CSE565_PROFILE", "standard")
50 if PROFILE_NAME not in PROFILES:
51     raise RuntimeError(f"Unknown CSE565_PROFILE={PROFILE_NAME!r}; choose one of {sorted(PROFILES)}.")
52 PROFILE = PROFILES[PROFILE_NAME]
53 API_MODE = os.getenv("CSE565_API_MODE", "simulated")
54 API_URL = os.getenv("CSE565_API_URL", "https://jsonplaceholder.typicode.com/posts")
55 SLO_RESPONSE_TIME_MS = float(os.getenv("CSE565_SLO_MS", "0"))
56 ARTIFACT_DIR = Path(os.getenv("CSE565_ARTIFACT_DIR", "locust_results/current"))
57 ARTIFACT_DIR.mkdir(parents=True, exist_ok=True)
58 LOG_LOCK = Semaphore()
59
60
61 @events.test_start.add_listener
62 def stop_after_requested_duration(environment, **kwargs: object) -> None:
63     """Use a gevent timer for repeatable CLI runs on the local Python runtime."""
64     raw_duration = os.getenv("CSE565_DURATION_SECONDS")
65     if raw_duration:
66         duration = float(raw_duration)
67         if duration <= 0:
68             raise RuntimeError("CSE565_DURATION_SECONDS must be greater than zero.")
69         gevent.spawn_later(duration, environment.runner.quit)
70
71
72 class SampleApplicationUser(User):
73     """One Locust user that repeatedly executes the six sample-workload tasks."""
74
75     # A wait avoids an unbounded tight loop and models think time between workflows.
76     wait_time = between(0.5, 1.0)
77
78     def on_start(self) -> None:
79         self._random = random.Random(id(self))
80
81     def _record(self, name: str, operation: Callable[[], int | str]) -> None:
82         started = time.perf_counter()
83         result_size = 0
84         exception: Exception | None = None

```

Figure 2: Locust workflow configuration for the performance-test workload.

3 Task 3: Test Execution Results

The three immediate-spawn tests were completed through Locust on October 4, 2026. Table 2 reproduces the aggregate row from each Locust report. Response-time values are in milliseconds.

RPS was monitored to measure completed-work throughput, response time was monitored to identify user-visible delay, and failures were monitored to distinguish a slow but completed task from an

exception or an invalid result. Using all three metrics is necessary because a higher RPS alone does not show whether individual tasks remain responsive or correct.

Table 2: Aggregate Locust results for the three baseline tests.

Users	Requests	Failures	Average (ms)	Median (ms)	Maximum (ms)	RPS
100	23,018	0	2.903	0.009	21.449	792.579
1,000	237,133	0	2.804	0.009	49.595	7,895.465
10,000	739,782	0	146.865	0.009	730.365	23,453.975

Table 3: Per-task Locust response-time results.

Load and task	Average (ms)	Median (ms)	RPS
100: Data Processing	0.167	0.057	132.119
100: Database Query	5.618	5.006	132.119
100: API Request	10.981	11.000	132.085
100: File I/O	0.482	0.074	132.085
100: Computation	0.026	0.009	132.085
100: Logging	0.146	0.022	132.085
1,000: Data Processing	0.079	0.057	1,316.771
1,000: Database Query	5.682	5.001	1,316.005
1,000: API Request	10.869	10.002	1,315.672
1,000: File I/O	0.144	0.074	1,315.672
1,000: Computation	0.012	0.009	1,315.672
1,000: Logging	0.040	0.021	1,315.672
10,000: Data Processing	0.062	0.057	4,024.620
10,000: Database Query	435.702	470.000	3,944.789
10,000: API Request	445.580	480.000	3,871.141
10,000: File I/O	0.121	0.071	3,871.141
10,000: Computation	0.010	0.009	3,871.141
10,000: Logging	0.039	0.021	3,871.141

4 Task 4: Analysis of Test Execution Results

The 100-user and 1,000-user immediate-spawn tests completed with zero recorded failures. At 100 users, the aggregate Locust report recorded 792.579 requests per second (RPS), and at 1,000 users it recorded 7,895.465 RPS. The simulated database query and API request had low response times at these two levels. For example, the database-query average was 5.618 ms at 100 users and 5.682 ms at 1,000 users;

the API-request average was 10.981 ms and 10.869 ms, respectively.

The 10,000-user test also completed with zero recorded failures. The aggregate RPS increased to 23,453.975, which shows that Locust continued to complete measured task requests. However, the response-time data changed substantially for the simulated database-query and API-request tasks. The database-query average increased to 435.702 ms with a 470.000 ms median and a 729.283 ms maximum. The API-request average increased to 445.580 ms with a 480.000 ms median and a 730.365 ms maximum. In comparison, the data-processing, computation, file-I/O, and logging task averages remained below 1 ms at the same load level.

These results show no reported execution failures, but they do show a substantial responsiveness degradation at 10,000 users for the database-query and API-request portions of the workflow. A response time near one-half second remains a completed response rather than a failure, but it is noticeably slower than the approximately 5–11 ms response times recorded at 100 and 1,000 users. Therefore, the system remained available during the 10,000-user test, while the affected workflow tasks became less responsive.

One limitation is that the workload is executed by the Locust user process itself. Consequently, the increased delay may reflect saturation of the Locust/gevent scheduling process in addition to pressure in the simulated application workload. The results identify the database-query and API-request path as the locations where the test observed degradation, but a separate application target process would be needed before attributing the full response-time increase solely to an application-service bottleneck.

5 Task 5: Improvement of Test Strategy

Task 4 showed an important gap in the original strategy: the 10,000-user test reported zero Locust exceptions even though the database-query and API-request tasks had become much slower. For Task 5, the same immediate 10,000-user workload was retained, but the observation period was extended from 30 to 60 seconds and a 500 ms response-time service-level objective (SLO) was added. A task that takes more than 500 ms is recorded as a performance failure. This is a meaningful improvement because it turns the response degradation already observed in Task 4 into a clear pass/fail result without changing the workload being tested.

The 500 ms limit was selected because a half-second delay is substantially slower than the 5–11 ms response times at 100 and 1,000 users and represents a noticeable delay in an interactive application. It is not an exception in the code; it is a performance failure based on the requirement that the task

remain responsive.

```
CSE565_PROFILE=standard CSE565_API_MODE=simulated \  
CSE565_DURATION_SECONDS=60 CSE565_SLO_MS=500 \  
.venv/bin/locust -f locustfile.py --headless \  
--users 10000 --spawn-rate 10000 --stop-timeout 30
```

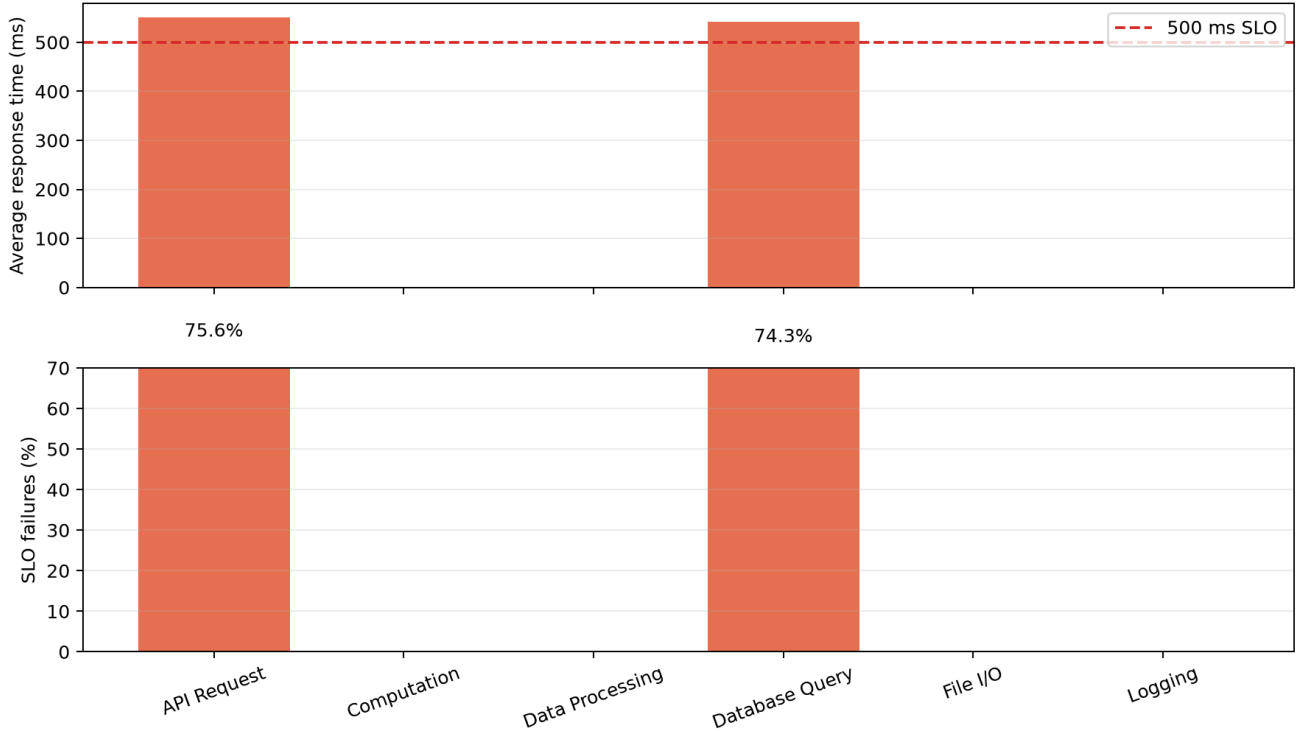
The 60-second test recorded 1,433,657 measured task requests and 358,325 SLO failures (24.99%). All SLO failures occurred in the same database-query and API-request paths that degraded in Task 4. The other four task types had zero failures. This outcome verifies that the Task 4 latency is not merely a small numerical change: under the stated response-time requirement, it is an observable performance failure.

Table 4: Task 5 10,000-user, 60-second SLO results.

Task	Requests	SLO failures	Failure rate	Average (ms)
API Request	237,984	179,989	75.63%	551.529
Database Query	240,105	178,336	74.27%	541.724
All other tasks	955,568	0	0.00%	< 1

Figure 3 makes the outcome easy to see: only the database-query and API-request tasks cross the 500 ms average-response-time line and receive SLO failures. The strategy did not identify a new code path beyond Task 4. Instead, it provided stronger evidence about the existing bottlenecks by showing that they violate a user-facing performance requirement. The result also shows a weakness of the test environment: Locust reported high local CPU utilization during the run, so a follow-up should use distributed Locust workers or a separate application target to confirm the exact capacity of the simulated service.

Task 5: 10,000 Immediate Users for 60 Seconds with a 500 ms SLO



Only the database-query and API-request tasks exceeded the 500 ms response-time SLO.

Figure 3: Task 5 results: 10,000 immediate users for 60 seconds with a 500 ms SLO.

6 Conclusion

The requested baseline tests show that the workflow completed without Locust-recorded execution failures at 100, 1,000, and 10,000 immediately spawned users. Performance was comparable at 100 and 1,000 users. At 10,000 users, the database-query and API-request tasks degraded from approximately 5–11 ms to approximately 436–446 ms on average, even though the tasks still completed. This distinguishes availability from acceptable responsiveness: a completed request is not necessarily a timely request.

The revised Task 5 strategy gave the Task 4 degradation a meaningful pass/fail interpretation. With a 500 ms response-time SLO, the 10,000-user workload produced failures in approximately three-quarters of the API-request and database-query tasks, while all other tasks passed. Thus, the improved strategy did not discover a separate bottleneck, but it clearly establishes that the two already observed slow paths fail an interactive-performance requirement under sustained high load.

References

Locust. (n.d.). *Your first test*. Locust documentation. Retrieved October 4, 2026, from <https://docs.locust.io/en/stable/quickstart.html>